

ETEC Vasco Antônio Venchiarutti

Luigi Pozzani de Souza 2C

Murilo Dias da Silva 2C

Nicolas Camargo Costa Ceccato 2C

Pesquisa

-

Criptografia em PHP

1. O que é a criptografia em PHP, qual seu objetivo e funções

A criptografia em PHP é um conjunto de técnicas utilizadas para proteger informações importantes em uma aplicação. Seu principal objetivo é impedir que pessoas não autorizadas consigam ler ou modificar dados, garantindo mais segurança para sistemas e usuários. A criptografia é muito utilizada em sites, sistemas de login, bancos, lojas virtuais e qualquer aplicação que trabalhe com informações sensíveis.

O PHP oferece diversos recursos para trabalhar com criptografia. Um dos mais importantes é a função `password_hash()`, utilizada para transformar senhas em um hash seguro antes de armazená-las no banco de dados. Quando o usuário faz login, a função `password_verify()` compara a senha digitada com o hash armazenado, sem que a senha original precise ser salva. Esse método aumenta bastante a segurança das aplicações.

Além da proteção de senhas, o PHP também permite criptografar e descriptografar dados utilizando extensões como OpenSSL e Sodium (Libsodium). Essas ferramentas são usadas para proteger arquivos, mensagens, chaves e outras informações que precisam permanecer confidenciais durante o armazenamento ou a transmissão pela internet. A biblioteca Sodium é considerada uma das opções mais modernas e seguras disponíveis no PHP.

A criptografia é fundamental para a segurança das aplicações desenvolvidas em PHP, pois ajuda a proteger os dados dos usuários contra acessos indevidos e ataques virtuais. Por isso, utilizar as funções e bibliotecas de segurança oferecidas pela linguagem é uma boa prática no desenvolvimento de sistemas confiáveis.

2. Métodos e exemplos

No PHP, a criptografia se divide em três grandes categorias, **Hashing**, **Criptografia Simétrica** e **Criptografia Assimétrica**, serão abordados um por um:

2.1 Hashing

O Hashing é uma transformação de mão única (unidirecional), o que significa que depois que uma informação é transformada em hash, não existe uma função de "descriptografar" que simplesmente faça o caminho inverso e revele a informação original, o que não classifica ele exatamente como criptografia, pois, imagine um documento, ao transformar ele em hash, não há como reverter em documento denovo para checar.

Funciona da seguinte maneira: Uma função hash recebe uma entrada de um tamanho variável de caracteres, realiza operações matemáticas e produz uma saída de tamanho determinado, ficando sujeito à quantidade de bits da função de hashing utilizada. Pequenas alterações na entrada normalmente produzem resultados muito diferentes, o que torna o resultado nada previsível e adivinhável. Hashes são usados para verificação de integridade e, com funções apropriadas, para armazenamento seguro de senhas.

Aqui vão alguns tipos de hashing:

- Família Message Digest
 - Muito popular antigamente, mas hoje obsoleto, por que ocasionam em colisões e possuem vulnerabilidades;
 - Contou com 6 versões, indo do MD2 ao MD5 (o MD6 acabou por ser eliminado por alguns problemas técnicos), todas as versões foram criadas por Ron Rivest, professor do MIT;

- Todos possuem saída de 128 bits (32 caracteres hexadecimais).

- Secure Hash Algorithm (SHA)

- Amplamente utilizado até os dias de hoje, se divide em subfamílias, sendo elas SHA-0, SHA-1, SHA-2 e SHA-3;
- A mais utilizada é o SHA-2, apesar de sua sucessora;
- Os algoritmos mais populares do SHA-2 são:
 - SHA-256, que é otimizado para processadores de 32 bits, e produz um output de 256 bits;
 - SHA-512, otimizado para processadores de 64 bits, e produz um output de 512 bits.
- Não são bons para armazenar senhas, principalmente pelo fato de serem muito rápidos.

- Argon2id

- Muito utilizado, referência na indústria para proteção de senhas, sendo o vencedor do *Password Hashing Competition* de 2015;
- Completamente personalizável, pode definir um output entre 4 e 2^{32} bits, pode definir o número de passagens pelo algoritmo, um limite de RAM e de threads.

- bcrypt

- Também muito utilizado para senhas, foi feito para ser lento de propósito, protegendo de ataques do tipo brute force (força bruta);
- O diferencial do bcrypt, é que além de gerar o hash, gera outro resultado chamado salt, que é único para cada usuário. Imagine uma

senha, o bcrypt pega ela e transforma em dois fatores, o hash e o salt, esses dois fatores são combinados, supondo que 100 pessoas usem a mesma senha, um invasor pode até descobrir o hash, mas cada pessoa vai ter um salt diferente somado com esse hash, o que cria uma espécie de segunda autenticação que é difícil de burlar a não ser por brute force (extremamente demorado).

2.2 Criptografia Simétrica

A criptografia simétrica é o método onde uma única chave secreta é utilizada tanto para criptografar os dados quanto para descriptografar os dados, pense como uma porta, precisa da chave para abrir, e da chave para fechar.

As vantagens são a facilidade e rapidez, exige muito pouco poder de processamento, mas carrega a desvantagem de ter que proteger muito a chave, se alguém interceptar essa chave de qualquer forma que seja, toda a segurança não existe mais.

Extensões mais utilizadas:

- OpenSSL
 - É a opção mais tradicional, é usada mais pelo legado do que por qualquer outro fator, ainda se prova fiel nos dias atuais.
- Sodium (LibSodium)
 - Nativa do PHP desde a versão 7.2, é considerada a melhor opção atualmente, principalmente por ser fácil de configurar e usar.

2.3 Criptografia Assimétrica

Ligeiramente mais complexa que a simétrica, ao invés de usar uma chave só que é secreta, você tem duas, uma chave pública que é usada pra codificar os dados, que todo mundo pode ter acesso, e uma chave privada, que só o dono tem, usada para decifrar as mensagens ou criar assinaturas digitais.

Imagine uma pessoa que seus colegas querem te mandar mensagens secretas, invés de dar a eles o cadeado com a chave, você dá apenas o cadeado, ele insere a

mensagem, tranca ela com o cadeado, e te envia, ele não pode abrir o cadeado que ele mesmo trancou pelo fato de não ter a chave, só você tem a chave.

Esse tipo de criptografia também é majoritariamente utilizado no OpenSSL e Sodium, essas extensões suportam os dois tipos de criptografia.